Blacklist

Interface Design Description

Eclipse Arrowhead 5.2.2 | support system | Revision 2026-11-09 | REVIEW DRAFT

Purpose and document structure

This document consolidates the system interface designs. Detailed Java service/transport IDDs are retained as separate appendices, followed by available Go and Rust interface descriptions. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring. 7 detailed Java IDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-Blacklist-go.

Blacklist - Go 5.2.2

Standalone support system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/Blacklist. Entry point: ./cmd/blacklist. Module: arrowhead/ah5-support-go/blacklist. See DEPLOYMENT.md for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and UPSTREAM-README.md. The dedicated launcher selects only this support-system role. The formerly shared internal runtime and regression tests are copied locally, not linked from another tree. Files for common helpers and other route implementations remain in that private runtime; this packaging does not claim new functionality or Java parity. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

Listening port

Built-in default 127.0.0.1:8464 (Arrowhead default). The shipped config.json, compose.yaml and Dockerfile use the same port on all interfaces ("listen": "0.0.0.0:8464"; 8540-8555 layout before 2026-11-09). Override with "listen" in the configuration file, SERVER_PORT=N or -server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Blacklist	Create/query/revoke, active-time checks, lookup	Golden tests for normalization, historical reactivation and query-mode edge cases

Detailed implementation evidence

1 native Go IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-Blacklist-rust.

Blacklist (Rust)

Arrowhead 5.2.2 Blacklist support system. Entry point src/main.rs is built as binary blacklist by the shared crate in ../ah5-support-runtime-rust (arrowhead-support-rust).

- examples/blacklist.json - example configuration (master copy: ../ah5-support-runtime-rust/examples/)

- documentation/ - SysD/SysDD/IDD/SD sources (symlink into ../ah5-support-runtime-rust/documentation/Blacklist, where the contract generator writes)

Run: cargo run -p arrowhead-support-rust --bin blacklist -- <config.json>

Listening port

Default 127.0.0.1:8464 (Arrowhead default, role blacklist). Override with "listen": "host:port" in the configuration, SERVER_PORT=N or --server.port N; the port overrides keep the configured host. See [LISTENING-PORTS.md](../LISTENING-PORTS.md) for precedence and all systems.

Documented behavior and remaining work

System	Implemented behavior	Remaining work
Blacklist	Create/revoke, expiry/check/lookup, filtered queries	Full normalization and historical query parity

Detailed implementation evidence

1 native Rust IDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-blacklistDiscovery-HTTP-IDD.tex
- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-blacklistDiscovery-MQTT-IDD.tex
- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-blacklistManagement-HTTP-IDD.tex
- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-blacklistManagement-MQTT-IDD.tex
- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-generalManagement-HTTP-IDD.tex
- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-generalManagement-MQTT-IDD.tex
- Java: Eclipse-Arrowhead-java/ah5-Blacklist-java-spring/documentation/v5.2.2/Blacklist-v5.2.2-sysMonitor-AMQP-IDD.tex
- Go: Eclipse-Arrowhead-go/ah5-Blacklist-go/documentation/Blacklist/Blacklist-HTTPS-IDD.tex
- Rust: Eclipse-Arrowhead-rust/ah5-support-runtime-rust/documentation/Blacklist/Blacklist-HTTPS-IDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

blacklistDiscovery HTTP/TLS/JSON

Interface Design Description

Service ID: *"blacklistDiscovery"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `blacklistDiscovery` service of the Blacklist system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation check . . . 4
2.2 operation lookup . . . 4

3 Data Models **6**
3.1 struct BlacklistEntry . . . 6
3.2 struct BlacklistEntryListResponse . . . 6
3.3 struct ErrorMessage . . . 6
3.4 Primitives . . . 6

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `blacklistDiscovery` service interface of the Blacklist system, which provides checking whether a system is blacklisted in the local cloud, and listing the requester's own active entries.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `blacklistDiscovery` service interface

This document provides the Interface Design Description IDD to the *blacklistDiscovery – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `blacklistDiscovery` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `blacklist`, port 8464 and base path `/blacklist`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`check`	GET	`/check/{systemName}`
`lookup`	GET	`/lookup`

2 Service Operations

The interfaces of the `blacklistDiscovery` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `blacklistDiscovery` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 GET /blacklist/check/*{systemName}*

Operation: check
Input: String
Output: Boolean

Returns true if the named system has an active blacklist entry. The system name fills the `{systemName}` placeholder of the registered path.

Path parameter: *systemName*.

A successful call returns `200 OK` with the output in the body.

```
GET /blacklist/check/<systemName> HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 1: An check invocation.

```
true
```

Listing 2: An check response.

2.2 GET /blacklist/lookup

Operation: lookup
Output: BlacklistEntryListResponse

Returns the active entries about the requester.

A successful call returns `200 OK` with the output in the body.

```
GET /blacklist/lookup HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An lookup invocation.

```
{
  "entries": [
    {
      "systemName": "TemperatureSensor",
      "createdBy": "Sysop",
```

```
      "revokedBy": "string",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z",
      "reason": "Compromised certificate",
      "expiresAt": "2026-10-22T12:00:00Z",
      "active": true
    }
  ],
  "count": 1
}
```


Listing 4: An lookup response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.4, which are used to represent things like hashes, identifiers and texts.

3.1 struct BlacklistEntry

A blacklist entry.

Object Field	Value Type	Description
"systemName"	String	The blacklisted system.
"createdBy"	String	System that created the entry.
"revokedBy"	String	System that inactivated the entry, if any.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.
"reason"	String	Reason of the blacklisting.
"expiresAt"	DateTime	Expiry of the entry; no value means permanent.
"active"	Boolean	Whether the entry is in force.

3.2 struct BlacklistEntryListResponse

List of blacklist entries.

Object Field	Value Type	Description
"entries"	Array<BlacklistEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “blacklistDiscovery – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-blacklistDiscovery-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

blacklistDiscovery MQTT/TLS/JSON

Interface Design Description

Service ID: *"blacklistDiscovery"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `blacklistDiscovery` service of the Blacklist system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation check . . . 4
2.2 operation lookup . . . 4

3 Data Models **5**
3.1 struct MqttRequestTemplate . . . 5
3.2 struct MqttResponseTemplate . . . 5
3.3 struct ErrorMessage . . . 5
3.4 struct BlacklistEntry . . . 5
3.5 struct BlacklistEntryListResponse . . . 6
3.6 Primitives . . . 6

4 References **7**

5 Revision History **8**
5.1 Amendments . . . 8
5.2 Quality Assurance . . . 8

1 Overview

This document describes the `blacklistDiscovery` service interface of the Blacklist system over MQTT, which provides checking whether a system is blacklisted in the local cloud, and listing the requester's own active entries.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `blacklistDiscovery` service interface

This document provides the Interface Design Description IDD to the *blacklistDiscovery – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/blacklist/`. The standalone deployment sets `mqtt.api.enabled=false`, so the MQTT interfaces are not registered there.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`check`	`arrowhead/blacklist/check`
`lookup`	`arrowhead/blacklist/lookup`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/blacklist/check`

Operation: **check**
Input: **String**
Output: **Boolean**

Returns true if the named system has an active blacklist entry. The system name fills the `{systemName}` placeholder of the registered path.

```
"TemperatureSensor"
```

Listing 3: Payload of an check request.

2.2 PUBLISH `arrowhead/blacklist/lookup`

Operation: **lookup**
Output: **BlacklistEntryListResponse**

Returns the active entries about the requester.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct BlacklistEntry

A blacklist entry.

Object Field	Value Type	Description
"systemName"	String	The blacklisted system.
"createdBy"	String	System that created the entry.
"revokedBy"	String	System that inactivated the entry, if any.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.
"reason"	String	Reason of the blacklisting.
"expiresAt"	DateTime	Expiry of the entry; no value means permanent.
"active"	Boolean	Whether the entry is in force.

3.5 struct BlacklistEntryListResponse

List of blacklist entries.

Object Field	Value Type	Description
"entries"	Array<BlacklistEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.6 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “blacklistDiscovery – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-blacklistDiscovery-SD.pdf.

[6] ——, “blacklistDiscovery HTTP/TLS/JSON – Interface Design Description (Blacklist),” Eclipse Arrowhead, IDD 5.2.2, 2026, blacklist-v5.2.2-blacklistDiscovery-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

blacklistManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"blacklistManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `blacklistManagement` service of the Blacklist system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation query . . . 4
2.2 operation create . . . 5
2.3 operation remove . . . 5

3 Data Models **7**
3.1 struct BlacklistCreateListRequest . . . 7
3.2 struct BlacklistCreateRequest . . . 7
3.3 struct BlacklistEntry . . . 7
3.4 struct BlacklistEntryListResponse . . . 7
3.5 struct BlacklistQueryRequest . . . 7
3.6 struct ErrorMessage . . . 8
3.7 struct Page . . . 8
3.8 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `blacklistManagement` service interface of the Blacklist system, which provides operator management of the blacklist: adding systems with a reason and expiry, querying entries and inactivating them.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `blacklistManagement` service interface

This document provides the Interface Design Description IDD to the *blacklistManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `blacklistManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `blacklist`, port 8464 and base path `/blacklist/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`query`	POST	`/query`
`create`	POST	`/create`
`remove`	DELETE	`/remove`

2 Service Operations

The interfaces of the `blacklistManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `blacklistManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /blacklist/mgmt/query

Operation: query
Input: BlacklistQueryRequest
Output: BlacklistEntryListResponse

Returns a page of entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /blacklist/mgmt/query HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "systemNames": [
    "TemperatureSensor"
  ],
  "mode": "ACTIVES",
  "issuers": [
    "string"
  ],
  "revokers": [
    "string"
  ],
  "reason": "Compromised certificate",
  "alivesAt": "2026-10-22T12:00:00Z"
}
```

Listing 1: An query invocation.

```
{
  "entries": [
    {
      "systemName": "TemperatureSensor",
      "createdBy": "Sysop",
      "revokedBy": "string",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z",
```

```
      "reason": "Compromised certificate",
      "expiresAt": "2026-10-22T12:00:00Z",
      "active": true
    }
  ],
  "count": 1
}
```

Listing 2: An query response.

2.2 POST /blacklist/mgmt/create

Operation: create
Input: BlacklistCreateListRequest
Output: BlacklistEntryListResponse

Adds blacklist entries with reason and optional expiry.
A successful call returns `200 OK` with the output in the body.

```
POST /blacklist/mgmt/create HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "entities": [
    {
      "systemName": "TemperatureSensor",
      "expiresAt": "2026-10-22T12:00:00Z",
      "reason": "Compromised certificate"
    }
  ]
}
```

Listing 3: An create invocation.

```
{
  "entries": [
    {
      "systemName": "TemperatureSensor",
      "createdBy": "Sysop",
      "revokedBy": "string",
      "createdAt": "2026-10-22T12:00:00Z",
      "updatedAt": "2026-10-22T12:00:00Z",
      "reason": "Compromised certificate",
      "expiresAt": "2026-10-22T12:00:00Z",
      "active": true
    }
  ],
  "count": 1
}
```

Listing 4: An create response.

2.3 DELETE /blacklist/mgmt/remove?names=*{names}*

Operation: remove
Input: Array<String>

Inactivates the entries of the named systems.
Query parameters: *names* (List<String>, required). List values are repeated or comma-separated.

A successful call returns `200 OK`.

```
DELETE /blacklist/mgmt/remove?names=<names> HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 5: An remove invocation.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.8, which are used to represent things like hashes, identifiers and texts.

3.1 struct BlacklistCreateListRequest

Entries to add to the blacklist.

Object Field	Value Type	Description
"entities"	Array<BlacklistCreateRequest>	Entries to create.

3.2 struct BlacklistCreateRequest

One system to blacklist.

Object Field	Value Type	Description
"systemName"	String	System to blacklist.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"reason"	String	Reason of the blacklisting.

3.3 struct BlacklistEntry

A blacklist entry.

Object Field	Value Type	Description
"systemName"	String	The blacklisted system.
"createdBy"	String	System that created the entry.
"revokedBy"	String	System that inactivated the entry, if any.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.
"reason"	String	Reason of the blacklisting.
"expiresAt"	DateTime	Expiry of the entry; no value means permanent.
"active"	Boolean	Whether the entry is in force.

3.4 struct BlacklistEntryListResponse

List of blacklist entries.

Object Field	Value Type	Description
"entries"	Array<BlacklistEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct BlacklistQueryRequest

Paged filter for blacklist entries.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"systemNames"	Array<String>	System names to match.
"mode"	String	Which entries to return: `ALL`, `ACTIVES` or `INACTIVES`.
"issuers"	Array<String>	Systems that created the entries.
"revokers"	Array<String>	Systems that inactivated the entries.
"reason"	String	Reason of the blacklisting.
"alivesAt"	DateTime	Only entries in force at this time.

3.6 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.7 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.8 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “blacklistManagement – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-blacklistManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

blacklistManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"blacklistManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `blacklistManagement` service of the Blacklist system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation create . . . 4
2.2 operation query . . . 4
2.3 operation remove . . . 5

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct BlacklistCreateListRequest . . . 6
3.5 struct BlacklistCreateRequest . . . 6
3.6 struct BlacklistEntry . . . 7
3.7 struct BlacklistEntryListResponse . . . 7
3.8 struct BlacklistQueryRequest . . . 7
3.9 struct Page . . . 7
3.10 Primitives . . . 8

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `blacklistManagement` service interface of the Blacklist system over MQTT, which provides operator management of the blacklist: adding systems with a reason and expiry, querying entries and inactivating them.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `blacklistManagement` service interface

This document provides the Interface Design Description IDD to the *blacklistManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/blacklist/management/`. The standalone deployment sets `mqtt.api.enabled=false`, so the MQTT interfaces are not registered there.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`create`	`arrowhead/blacklist/management/create`
`query`	`arrowhead/blacklist/management/query`
`remove`	`arrowhead/blacklist/management/remove`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/blacklist/management/create`

Operation: create
Input: BlacklistCreateListRequest
Output: BlacklistEntryListResponse

Adds blacklist entries with reason and optional expiry.

```
{
  "entities": [
    {
      "systemName": "TemperatureSensor",
      "expiresAt": "2026-10-22T12:00:00Z",
      "reason": "Compromised certificate"
    }
  ]
}
```

Listing 3: Payload of an create request.

2.2 PUBLISH `arrowhead/blacklist/management/query`

Operation: query
Input: BlacklistQueryRequest
Output: BlacklistEntryListResponse

Returns a page of entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "systemNames": [
    "TemperatureSensor"
  ],
  "mode": "ACTIVES",
  "issuers": [
    "string"
  ],
  "revokers": [
    "string"
  ],
  "reason": "Compromised certificate",
  "alivesAt": "2026-10-22T12:00:00Z"
}
```

Listing 4: Payload of an query request.

2.3 PUBLISH `arrowhead/blacklist/management/remove`

Operation: remove
Input: Array<String>

Inactivates the entries of the named systems.

```
[
  "TemperatureSensor"
]
```

Listing 5: Payload of an remove request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct BlacklistCreateListRequest

Entries to add to the blacklist.

Object Field	Value Type	Description
"entities"	Array<BlacklistCreateRequest>	Entries to create.

3.5 struct BlacklistCreateRequest

One system to blacklist.

Object Field	Value Type	Description
"systemName"	String	System to blacklist.
"expiresAt"	DateTime	UTC expiry time; no value means no expiry.
"reason"	String	Reason of the blacklisting.

3.6 struct BlacklistEntry

A blacklist entry.

Object Field	Value Type	Description
"systemName"	String	The blacklisted system.
"createdBy"	String	System that created the entry.
"revokedBy"	String	System that inactivated the entry, if any.
"createdAt"	DateTime	UTC time of creation.
"updatedAt"	DateTime	UTC time of the last change.
"reason"	String	Reason of the blacklisting.
"expiresAt"	DateTime	Expiry of the entry; no value means permanent.
"active"	Boolean	Whether the entry is in force.

3.7 struct BlacklistEntryListResponse

List of blacklist entries.

Object Field	Value Type	Description
"entries"	Array<BlacklistEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.8 struct BlacklistQueryRequest

Paged filter for blacklist entries.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"systemNames"	Array<String>	System names to match.
"mode"	String	Which entries to return: `ALL`, `ACTIVES` or `INACTIVES`.
"issuers"	Array<String>	Systems that created the entries.
"revokers"	Array<String>	Systems that inactivated the entries.
"reason"	String	Reason of the blacklisting.
"alivesAt"	DateTime	Only entries in force at this time.

3.9 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.10 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
Boolean	JSON Boolean, `true` or `false`.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “blacklistManagement – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-blacklistManagement-SD.pdf.

[6] ——, “blacklistManagement HTTP/TLS/JSON – Interface Design Description (Blacklist),” Eclipse Arrowhead, IDD 5.2.2, 2026, blacklist-v5.2.2-blacklistManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement HTTP/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the HTTP/1.1 protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the Blacklist system.

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct ErrorMessage . . . 6
3.2 struct KeyValues . . . 6
3.3 struct LogEntry . . . 6
3.4 struct LogEntryListResponse . . . 6
3.5 struct LogRequest . . . 6
3.6 struct Page . . . 7
3.7 Primitives . . . 7

4 References **8**

5 Revision History **9**
5.1 Amendments . . . 9
5.2 Quality Assurance . . . 9

1 Overview

This document describes the `generalManagement` service interface of the Blacklist system, which provides general management of the Blacklist system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	HTTP	1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. For further details about how this service is meant to be used, please consult that document.

The rest of this document describes how to realize the `generalManagement` service interface in detail. Both in terms of its operations (Section 2) and its information model (Section 3).

1.1 Interface registration

The service is registered in the ServiceRegistry with the interface template `generic_https`, policy `CERT_AUTH`, access address `blacklist`, port 8464 and base path `/blacklist/general/mgmt`. The `operations` property of the interface lists the operations below with their method and path relative to the base path (path parameters are placeholders such as `{id}` that the consumer fills in):

Operation	Method	Path
`get-log`	POST	`/logs`
`get-config`	GET	`/get-config`

2 Service Operations

The interfaces of the `generalManagement` service, its operations, data models and implementation are provided below. In particular, each subsection first names the HTTP method and path used to call the operation, after which it names the abstract operation from the `generalManagement` SD document, as well as input and output types.

All operations require the requester's identity, presented according to the provider's authentication policy: the client certificate (mutual TLS) and, with the `outsourced` policy, `Authorization: Bearer IDENTITY-TOKEN//<token>`; systems holding the Authentication secret key send `Bearer AUTHENTICATOR-KEY//<system>//<hash>`, and with the `declared` policy `Bearer SYSTEM//<system>` is sent. Request and response bodies are `application/json` unless stated otherwise. Errors are answered with an ErrorMessage body and the status codes `400 Bad Request` (invalid parameter), `401 Unauthorized` (missing or invalid identity), `403 Forbidden` (not permitted, e.g. by the management, authorization or blacklist filter), `404 Not Found`, `408 Request Timeout`, `423 Locked`, `500 Internal Server Error` and `503 Service Unavailable` (a system it depends on failed).

2.1 POST /blacklist/general/mgmt/logs

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

A successful call returns `200 OK` with the output in the body.

```
POST /blacklist/general/mgmt/logs HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
Content-Type: application/json

{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 1: An get-log invocation.

```
{
  "entries": [
    {
      "logId": "42",
      "entryDate": "2026-10-22T12:00:00Z",
      "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService",
      "severity": "INFO",
      "message": "Service instance registered: TemperatureSensor|temperatureReading|1.0.0"
    }
  ],
  "count": 1
}
```

Listing 2: An get-log response.

2.2 GET /blacklist/general/mgmt/get-config?keys=*{keys}*

Operation: get-config
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

Query parameters: *keys* (List<String>, optional). List values are repeated or comma-separated.

A successful call returns `200 OK` with the output in the body.

```
GET /blacklist/general/mgmt/get-config?keys=<keys> HTTP/1.1
Host: blacklist:8464
Authorization: Bearer IDENTITY-TOKEN//<token>
```

Listing 3: An get-config invocation.

```
{
  "map": {
    "server.port": "8443"
  }
}
```

Listing 4: An get-config response.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Note that each subsection, which describes one type of object, begins with the *struct* keyword, which is meant to denote a JSON Object that must contain certain fields, or names, with values conforming to explicitly named types. Fields that are not needed may be omitted or `null` in requests; empty fields are omitted in responses. As a complement to the primary types defined in this section, there is also a list of secondary types in Section 3.7, which are used to represent things like hashes, identifiers and texts.

3.1 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.2 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.3 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.4 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.5 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.6 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.7 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available. Note that the official standard is defined in terms of parsing rules, while this list only concerns syntactic information. Furthermore, the Object and Array types are given optional generic type parameters, which are used in this document to signify when pair values or elements are expected to conform to certain types.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] R. Fielding, M. Nottingham, and J. Reschke, “Http/1.1,” IETF, RFC 9112, 2022.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-generalManagement-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

generalManagement MQTT/TLS/JSON

Interface Design Description

Service ID: *"generalManagement"*

Abstract
This document describes the MQTT protocol with TLS payload security and JSON payload encoding variant of the `generalManagement` service of the Blacklist system.

Contents

1 Overview **3**

2 Service Operations **4**
2.1 operation get-log . . . 4
2.2 operation get-config . . . 4

3 Data Models **6**
3.1 struct MqttRequestTemplate . . . 6
3.2 struct MqttResponseTemplate . . . 6
3.3 struct ErrorMessage . . . 6
3.4 struct KeyValues . . . 6
3.5 struct LogEntry . . . 6
3.6 struct LogEntryListResponse . . . 7
3.7 struct LogRequest . . . 7
3.8 struct Page . . . 7
3.9 Primitives . . . 7

4 References **9**

5 Revision History **10**
5.1 Amendments . . . 10
5.2 Quality Assurance . . . 10

1 Overview

This document describes the `generalManagement` service interface of the Blacklist system over MQTT, which provides general management of the Blacklist system: reading its log entries and its configuration.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	MQTT	3.1.1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `generalManagement` service interface

This document provides the Interface Design Description IDD to the *generalManagement – Service Description* document [5]. The operations, their semantics and the data models are the same as in the HTTP/TLS/JSON IDD [6]; this document specifies the MQTT binding.

The MQTT interface is offered only when the system runs with `mqtt.api.enabled=true`; it is then registered with the interface template `generic_mqtts` (`generic_mqtt` without TLS), the broker address and port (`mqtt.broker.address`, `mqtt.broker.port`) and the base topic `arrowhead/blacklist/general/management/`. The standalone deployment sets `mqtt.api.enabled=false`, so the MQTT interfaces are not registered there.

2 Service Operations

The provider subscribes to one topic per operation, the base topic followed by the operation name:

Operation	Request topic
`get-log`	`arrowhead/blacklist/general/management/get-log`
`get-config`	`arrowhead/blacklist/general/management/get-config`

A consumer publishes a request envelope (MqttRequestTemplate) to the topic and receives the answer (MqttResponseTemplate) on its own `responseTopic`. The `authentication` field carries the requester's identity according to the provider's authentication policy: `IDENTITY-TOKEN//<token>` (outsourced or internal policy), `SYSTEM//<name>` (declared policy) or the requester's X.509 certificate (certificate policy). The `status` of the response uses the HTTP status code numbers (200, 201, 204, 400, 401, 403, 404, 408, 423, 500, 503); on error the payload is an ErrorMessage.

```
{
  "traceId": "a1b2c3",
  "authentication": "IDENTITY-TOKEN//<token>",
  "responseTopic": "arrowhead/response/TemperatureDisplay/a1b2c3",
  "qosRequirement": 1,
  "params": {},
  "payload": {}
}
```

Listing 1: Request envelope.

```
{
  "status": 200,
  "traceId": "a1b2c3",
  "receiver": "TemperatureDisplay",
  "payload": {}
}
```

Listing 2: Response envelope.

2.1 PUBLISH `arrowhead/blacklist/general/management/get-log`

Operation: get-log
Input: LogRequest
Output: LogEntryListResponse

Returns a page of the system's log entries matching the filter.

```
{
  "pagination": {
    "page": 0,
    "size": 10,
    "direction": "ASC",
    "sortField": "createdAt"
  },
  "from": "2026-10-22T12:00:00Z",
  "to": "2026-10-22T12:00:00Z",
  "severity": "INFO",
  "logger": "eu.arrowhead.serviceregistry.service.ServiceDiscoveryService"
}
```

Listing 3: Payload of an get-log request.

2.2 PUBLISH `arrowhead/blacklist/general/management/get-config`

Operation: get-config
Input: Array<String>
Output: KeyValues

Returns the effective values of the requested configuration keys. Only the public keys of the system are answered; other keys, including all secrets, are ignored.

```
[
  "string"
]
```

Listing 4: Payload of an get-config request.

3 Data Models

Here, all data objects that can be part of the service calls associated with this service are listed in alphabetic order. Each *struct* denotes a JSON Object with the named fields; fields that are not needed may be omitted.

3.1 struct MqttRequestTemplate

The request envelope.

Object Field	Value Type	Description
"traceId"	String	Identifier chosen by the requester, returned in the response.
"authentication"	String	Identity of the requester (see Section 2).
"responseTopic"	String	Topic on which the response is published.
"qosRequirement"	Number	MQTT QoS of the response (0, 1 or 2).
"params"	Object<String>	Optional parameters (the query parameters of the HTTP binding).
"payload"	Object	The operation input.

3.2 struct MqttResponseTemplate

The response envelope.

Object Field	Value Type	Description
"status"	Number	Status code.
"traceId"	String	The trace identifier of the request.
"receiver"	String	Name of the requester.
"payload"	Object	The operation output, or an ErrorMessage.

3.3 struct ErrorMessage

Object Field	Value Type	Description
"errorMessage"	String	Error text, if the event is an error.
"errorCode"	Number	
"exceptionType"	ExceptionType	
"origin"	String	

3.4 struct KeyValues

Configuration values of the system.

Object Field	Value Type	Description
"map"	Object<String>	The configuration keys and values; secrets are not returned.

3.5 struct LogEntry

One log entry.

Object Field	Value Type	Description
"logId"	String	Identifier of the log entry.
"entryDate"	DateTime	UTC time of the log entry.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.
"severity"	String	Log level, e.g. `INFO`, `WARN`, `ERROR`; as filter, the minimum level.
"message"	String	Text of the log entry or status message of the job.
"exception"	String	Stack trace or exception text of the log entry, if any.

3.6 struct LogEntryListResponse

List of log entries.

Object Field	Value Type	Description
"entries"	Array<LogEntry>	Matching records of the current page.
"count"	Number	Total number of matching records (all pages).

3.7 struct LogRequest

Paged filter for the log entries stored in the system's database.

Object Field	Value Type	Description
"pagination"	Page	Paging and sorting of the result.
"from"	DateTime	Return entries from this UTC time.
"to"	DateTime	Return entries until this UTC time.
"severity"	String	Minimum log level to return.
"logger"	String	Logger (Java class) that wrote the entry; as filter, its name.

3.8 struct Page

Paging and sorting; all fields optional.

Object Field	Value Type	Description
"page"	Number	Zero-based page number.
"size"	Number	Number of records per page (at most `max.page.size`, default 1000).
"direction"	String	Sort direction, `ASC` or `DESC`.
"sortField"	String	Field to sort by.

3.9 Primitives

As all messages are encoded using the JSON format [4], the following primitive constructs, part of that standard, become available.

Type	Description
Array	A JSON Array [4]; Array<A> has elements of type A.
DateTime	JSON String with an ISO 8601 UTC timestamp, e.g. `2026-10-22T12:00:00Z`.
ExceptionType	One of `ARROWHEAD`, `INVALID_PARAMETER`, `AUTH`, `FORBIDDEN`, `DATA_NOT_FOUND`, `TIMEOUT`, `LOCKED`, `INTERNAL_SERVER_ERROR`, `EXTERNAL_SERVER_ERROR` (JSON String).
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] A. Banks and R. Gupta, “Mqtt version 3.1.1,” OASIS, Standard, 2014.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “generalManagement – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-generalManagement-SD.pdf.

[6] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (Blacklist),” Eclipse Arrowhead, IDD 5.2.2, 2026, blacklist-v5.2.2-generalManagement-HTTP-IDD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): standalone deployment, configuration and security statements updated; missing primitive definitions added.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

sysMonitor AMQP/TLS/JSON

Interface Design Description

Service ID: *"sysMonitor"*

Abstract
This document describes the AMQP 0-9-1 protocol with TLS payload security and JSON payload encoding variant of the `sysMonitor` service of the Blacklist system (SysMonitor-AMQP-IDD-5.2.2).

Contents

1 Overview **3**
1.1 Interface registration . . . 3

2 Service Operations **4**
2.1 operation echo . . . 4
2.2 operation power . . . 4
2.3 operation status . . . 4

3 Data Models **6**
3.1 struct Observation . . . 6
3.2 Primitives . . . 6

4 References **7**

5 Revision History **8**
5.1 Amendments . . . 8
5.2 Quality Assurance . . . 8

1 Overview

This document describes the AMQP binding of the native `sysMonitor` service of Blacklist.

It's implemented using protocol, encoding as stated in below table:

Profile type	Type	Version
Transfer protocol	AMQP	0-9-1 [1]
Data encryption	TLS	1.3 [2], 1.2 [3]
Encoding	JSON	RFC 8259 [4]
Compression	N/A	-
Semantics	N/A	-
Ontology	N/A	-

Table 1: Communication and semantics details used for the `sysMonitor` service interface

This document provides the Interface Design Description IDD to the *sysMonitor – Service Description* document [5].

1.1 Interface registration

With `sysmonitor.enabled=true`, Blacklist registers the service at start-up with interface template `generic_amqps` (protocol `amqps`, policy `CERT_AUTH`; `generic_amqp` without TLS) and the following properties and metadata:

```
{
  "serviceDefinitionName": "sysMonitor",
  "version": "1.0.0",
  "metadata": {
    "contract": "SysMonitor-AMQP-IDD-5.2.2",
    "schemaVersion": "1.0",
    "publisherMode": "native",
    "observationTypes": [
      "echo",
      "power",
      "status"
    ],
    "delivery": "persistent, publisher-confirmed"
  },
  "interfaces": [
    {
      "templateName": "generic_amqps",
      "protocol": "amqps",
      "policy": "CERT_AUTH",
      "properties": {
        "accessAddresses": [
          "amqp-broker"
        ],
        "accessPort": 5671,
        "exchange": "arrowhead",
        "baseRoutingKey": "TestCloud.sysmonitor.Blacklist.Blacklist",
        "operations": [
          "echo",
          "power",
          "status"
        ]
      }
    }
  ]
}
```

Listing 1: Registered service instance (TestCloud).

2 Service Operations

The publisher is the `SysMonitorPublisher` of the common library, configured with `sysmonitor.broker.address`, `sysmonitor.broker.port`, `sysmonitor.local.cloud` (default: from the system certificate), `sysmonitor.publish.interval` and `sysmonitor.ssl.*` (default: the `server.ssl` stores). The publisher connects to the broker (default `amqp-broker`, port 5671) with TLS, verifies the broker host name and authenticates with its X.509 system certificate (SASL `EXTERNAL`; the certificate CN is the broker user name). Every publication interval it publishes each observation to the topic exchange `arrowhead` with the routing key `<baseRoutingKey>.<operation>`, where the base routing key is `<Cloud>.sysmonitor.<System>.<serviceInstance>`. Messages are persistent (delivery mode 2), `application/json`, carry the observation identifier as message id, and are publisher-confirmed (timeout 5 s); a failed cycle is retried with a new connection at the next interval. The broker's topic permissions allow the system to publish only below its own base routing key.

2.1 PUBLISH `<Cloud>.sysmonitor.Blacklist.Blacklist.echo`

Operation: echo
Output: Observation

Liveness: `status` (`alive` or `degraded`), `uptimeSeconds` and a `detail` text; quality `good` or `uncertain`.

2.2 PUBLISH `<Cloud>.sysmonitor.Blacklist.Blacklist.power`

Operation: power
Output: Observation

Power supply: `supplyType=line`, `state=available`; unit `1`.

2.3 PUBLISH `<Cloud>.sysmonitor.Blacklist.Blacklist.status`

Operation: status
Output: Observation

Process state: `state=running`, `health` (`healthy`/`degraded`) and `serviceCount`, the number of registered services.

```
{
  "schemaVersion": "1.0",
  "observationId": "Blacklist-echo-1792670400000-42",
  "providerSystem": "Blacklist",
  "serviceInstance": "Blacklist",
  "observationType": "echo",
  "sourceTime": "2026-10-22T12:00:00.000Z",
  "sequence": 42,
  "value": {
    "status": "alive",
    "uptimeSeconds": 3600,
    "detail": "Blacklist is running"
  },
  "quality": "good",
  "metadata": {
    "publisherMode": "native"
  }
}
```

Listing 2: An echo observation.

3 Data Models

3.1 struct Observation

Object Field	Value Type	Description
"schemaVersion"	String	`1.0`.
"observationId"	String	`<System>-<type>-<epoch ms>-<sequence>`.
"providerSystem"	String	The publishing system.
"serviceInstance"	String	Service instance, default the system name.
"observationType"	String	`echo`, `power`, `status`.
"sourceTime"	String	ISO 8601 UTC time with milliseconds.
"sequence"	Number	Sequence number per publisher.
"value"	Object	Observed value, see the operations.
"unit"	String	`1` for `power` and `status`; absent for `echo`.
"quality"	String	`good` or `uncertain`.
"metadata"	Object	`{"publisherMode": "native"}`.

3.2 Primitives

Type	Description
Number	JSON Number, an integer.
Object	JSON Object; Object<A> has string keys and values of type A.
String	JSON String.

4 References

[1] AMQP Working Group, “Amqp 0-9-1 specification,” AMQP Working Group, Tech. Rep., 2008.

[2] E. Rescorla, “The transport layer security (tls) protocol version 1.3,” IETF, RFC 8446, 2018.

[3] T. Dierks and E. Rescorla, “The transport layer security (tls) protocol version 1.2,” IETF, RFC 5246, 2008.

[4] T. Bray, “The javascript object notation (json) data interchange format,” IETF, RFC 8259, 2017.

[5] J. Delsing, “sysMonitor – Service Description (Blacklist),” Eclipse Arrowhead, SD 5.2.2, 2026, blacklist-v5.2.2-sysMonitor-SD.pdf.

5 Revision History

5.1 Amendments

No.	Date	Version	Subject of Amendments	Author
1	2026-10-22	5.2.2	Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace.	Jerker Delsing
2	2026-10-30	5.2.2	Checked against the primary Java source tree (Eclipse-Arrowhead-java): services, operations, paths and data models unchanged.	Claude Opus 5.5 (draft)

5.2 Quality Assurance

No.	Date	Version	Approved by
1		5.2.2	

1 Interface overview

HTTPS, mandatory mutual TLS, Arrowhead identity-token header and JSON request/response documents. General management is sysop/whitelist protected; enrollment endpoints use approved certificate fingerprints and profile/serial checks instead of ordinary identity tokens. Error responses contain errorMessage, errorCode, origin and timestamp. Exact DTO and status/error parity is not certified.

1.1 HTTPS interface operations

The listener is HTTPS with JSON by default; raw configuration and CRL responses use their declared media types. A request body is capped at 1 MiB. The table records implemented routes, not a claim of complete behavioral conformance.

Method	Path	Access
`GET`	`/blacklist/check/{systemName}`	identity
`GET`	`/blacklist/lookup`	identity
`POST`	`/blacklist/mgmt/create`	management
`DELETE`	`/blacklist/mgmt/remove`	management
`POST`	`/blacklist/mgmt/query`	management
`GET`	`/blacklist/monitor/echo`	certificate
`GET`	`/blacklist/general/mgmt/get-config`	management
`POST`	`/blacklist/general/mgmt/logs`	management

1.2 Important delimitations

This implementation is a development profile, not a certified interchangeable Java replacement. All systems use private JSON snapshots rather than the Java databases. MQTT application APIs, exhaustive DTO normalization and error parity, automatic data migration and production qualification remain outstanding.

- Time-bounded active/revoked records persist in JSON. Complete Java normalization, historical reactivation and query mode edge cases need golden-contract tests.

1 Blacklist

1.1 System overview

Native Rust development profile for local Java source `Blacklist/`. One independently configured executable, identity, port and state file per role. No Java, Go or Python process implements the service. Some essential operations are explicitly unavailable.

1.2 Design and trust boundaries

Tokio/Hyper/Axum with rustls enforces mutual TLS. Certificate chain, validity, local CN/profile and startup-configured optional CRLs are checked. CRLs need process restart to refresh; without CRLs no revocation enforcement is claimed. Tokens are verified with Authentication and bound to the TLS identity. Business calls require ConsumerAuthorization; management requires a verified sysop or explicit broad whitelist. CA provisioning instead applies Java-style named certificate-principal rights. Outbound authorities are pinned to expected CN/profile before credentials are sent, in addition to chain, SAN and time validation.

1.3 Persistent state

Independent locked atomic JSON replaces SQL. Clone/validate/fsync/rename commits prevent partially published local transactions. Rename is the commit point; directory-sync failure is an uncertain durable commit, not a rollback. Owner-only keys/password files are separate. Network effects are not distributed transactions; interrupted starts and bridge operations require reconciliation. No automatic Java state import, clustering or deployment is delivered.

1.4 Implemented behavior

Create/revoke, expiry/check/lookup, filtered queries

1.5 Open release gates

Full normalization and historical query parity. All systems remain development profiles, not certified interchangeable Java replacements. See `COMPATIBILITY.md` for detailed DTO fields and restrictions; the Java sources remain the authoritative contract.

1.6 Produced and consumed services

Advertised implemented HTTPS definitions: `blacklistDiscovery`, `blacklistManagement`, `echo`, `generalManagement`.

Authorities consumed as applicable: Authentication, ServiceRegistry, ConsumerAuthorization and explicitly configured pinned adapter/provider origins. Only implemented operations are registered. Service versions are 1.0.0 except Gatekeeper/GateTunnel 5.2.2. No AMQP sysMonitor.observe implementation is advertised.

1.7 HTTPS operations

JSON unless raw configuration bytes, PEM CRL or echo text. Bodies: 1 MiB; remote responses: 8 MiB. Header: `Bearer IDENTITY-TOKEN//token`. Unknown routes return 404, absent dependencies 503. Unported

operations return 501 after applicable access checks and are excluded from registration. Route presence is not semantic parity.

Method	Path	Access / status
GET	/blacklist/general/mgmt/get-config	management
POST	/blacklist/general/mgmt/logs	management
GET	/blacklist/monitor/echo	certificate
GET	/blacklist/check/{systemName}	identity
GET	/blacklist/lookup	identity
POST	/blacklist/mgmt/create	management
POST	/blacklist/mgmt/query	management
DELETE	/blacklist/mgmt/remove	management

Error envelope: errorMessage, errorCode, exceptionType, origin and timestamp. Full Java status, DTO and validation equivalence requires differential testing.

2 References and qualification

See `contracts.json` for local Java SHA256 hashes/mappings, `COMPATIBILITY.md` for DTO details and gaps, and `VERIFICATION.md` for tests actually run. No upstream release-tag provenance, complete replacement or real-cloud interoperability is asserted.